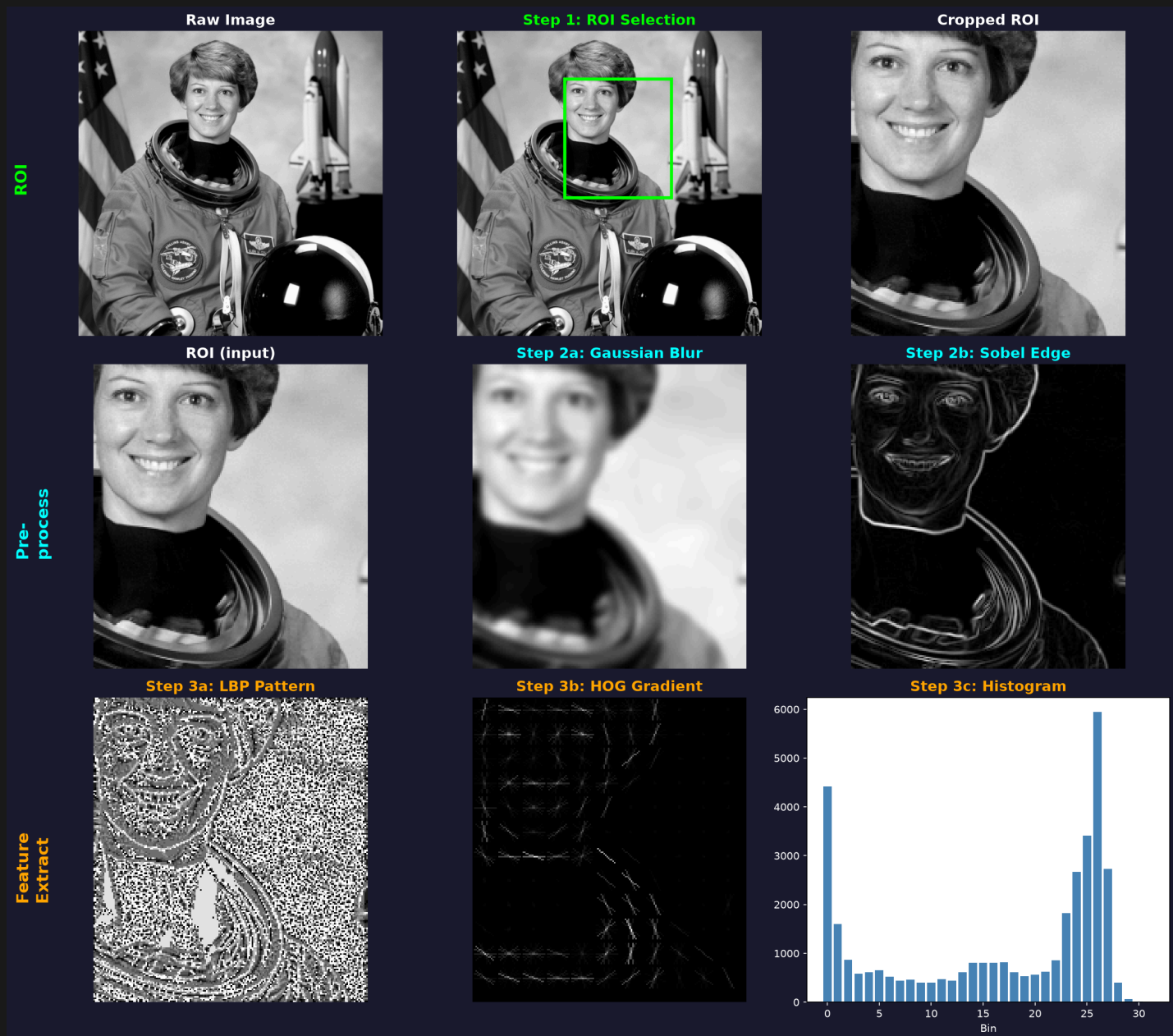


Explainable Feature Discovery

From Hand-Crafted to AI-Discovered Image Descriptors

Summer 2026 Intern Project



From Image to Feature Vector

Formalizing each pipeline step as a mathematical mapping:

Step	Notation	Mapping	Description
1. ROI Selection	$\mathbf{X} = \text{ROI}(I, r)$	$\mathbb{R}^{H \times W \times C} \rightarrow \mathbb{R}^{h \times w}$	Crop region r from image I , convert to grayscale
2. Pre-processing	$\mathbf{X}' = \mathcal{P}(\mathbf{X})$	$\mathbb{R}^{h \times w} \rightarrow \mathbb{R}^{h \times w}$	Stackable: $\mathcal{P} = p_k \circ \dots \circ p_1$
3. Feature Extraction	$\mathbf{f}_i = \mathcal{F}_i(\mathbf{X}')$	$\mathbb{R}^{h \times w} \rightarrow \mathbb{R}^{d_i}$	Multiple extractors applied in parallel
4. Classification	$\hat{y} = C(\mathbf{f}_1 \oplus \dots \oplus \mathbf{f}_n)$	$\mathbb{R}^{\sum d_i} \rightarrow \{1, \dots, K\}$	Classify on concatenated features (GAM / SVM)

Full pipeline: $I \xrightarrow{\text{ROI}} \mathbf{X} \xrightarrow{p_1 \circ \dots \circ p_k} \mathbf{X}' \xrightarrow{\mathcal{F}_1, \dots, \mathcal{F}_n} [\mathbf{f}_1 \oplus \dots \oplus \mathbf{f}_n] \xrightarrow{C} \hat{y}$

Research Question

$$I \xrightarrow{\text{ROI}} \mathbf{X} \xrightarrow{p_1 \circ \dots \circ p_k} \mathbf{X}' \xrightarrow{\mathcal{F}_1, \dots, \mathcal{F}_n, \mathcal{F}_{n+1}^*} [\mathbf{f}_1 \oplus \dots \oplus \mathbf{f}_n \oplus \mathbf{f}_{n+1}^*] \xrightarrow{\mathcal{C}} \hat{y}$$

Primary question: Can we automatically discover $\mathcal{F}_{n+1}^*$, a new image feature extractor expressed as an interpretable math formula, that is competitive with hand-crafted descriptors?

Background

Two directions in automated feature discovery

1. **Evolve from hand-designed basis elements:** Genetic Programming (GP) composes a fixed set of math operations into new descriptors
2. **Synthesize as programs:** Large Language Models (LLMs) write arbitrary code or mathematical expressions as features

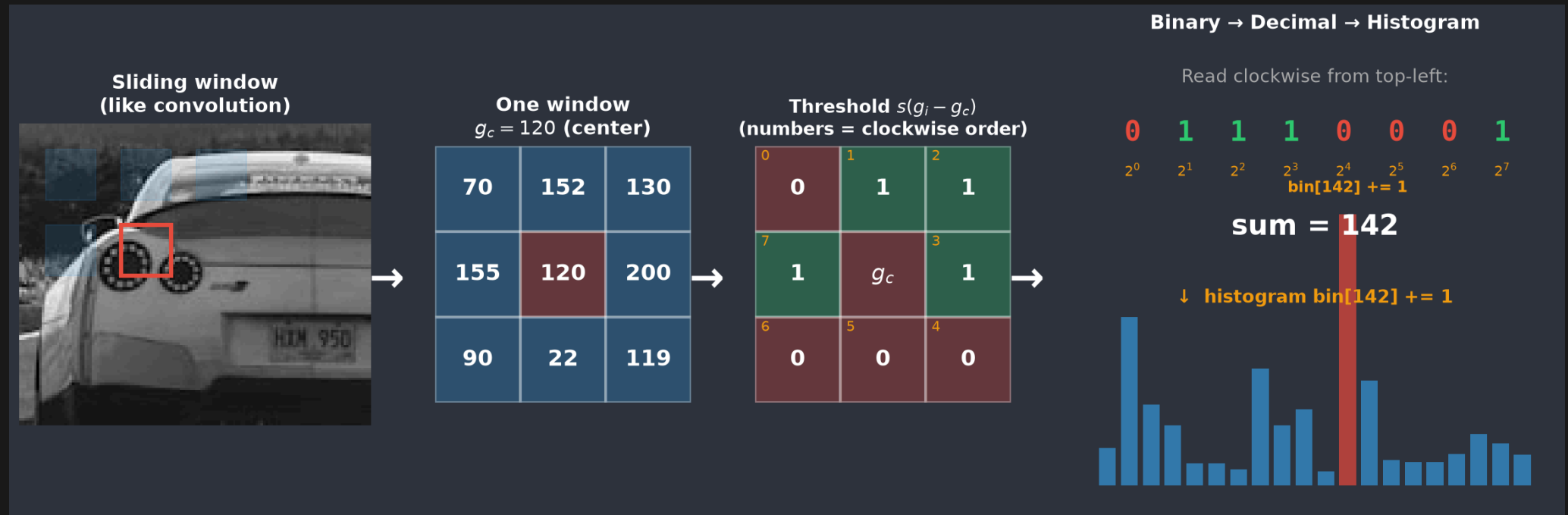
Approach 1: Search with Pools of Hand-Designed Basis Elements

Three levels of pools, from coarse to fine:

Level	Pool	Elements	Mapping
Pre-processing	$\mathcal{P}$ -pool	{ Gaussian, Sobel, Median, Laplacian, ... }	$\mathbb{R}^{h \times w} \rightarrow \mathbb{R}^{h \times w}$
Feature extractor	$\mathcal{F}$ -pool	{ HOG, LBP, SIFT, Histogram, ... }	$\mathbb{R}^{h \times w} \rightarrow \mathbb{R}^d$
Basis elements	$\mathcal{B}$ -pool	{ min, max, mean, stdev, +, -, $\times$, $\div$, ... }	scalar $\rightarrow$ scalar

Search objective: given these pools, find optimal compositions to maximize classification accuracy.
Existing work operates at different granularity levels (next slides).

Search Through Modified LBP (Al-Sahaf et al., IEEE TEVC 2017)



$$\text{LBP}_{p,r} = \sum_{i=0}^{p-1} s(g_i - g_c) \cdot 2^i, \quad s(x) = \begin{cases} 1 & x \geq 0 \\ 0 & \text{otherwise} \end{cases}$$

p = number of neighbors, r = radius of circular window, g_c = center pixel, g_i = i -th neighbor. Repeat for every pixel position, accumulate all codes into one histogram → feature vector $\mathbf{f} \in \mathbb{R}^d$. (Ojala et al., 1996)

Search Through Modified LBP (Al-Sahaf et al., IEEE TEVC 2017)

Keep the same LBP pipeline, **only replace the formula** inside the sliding window:

	LBP (hand-designed)	GP-cryptor (searched by GP)
Input	g_i, g_c (raw pixels)	$\min(w), \max(w), \text{mean}(w), \text{stdev}(w)$
Formula	$\sum_{i=0}^{p-1} s(g_i - g_c) \cdot 2^i$ (p bits, one per neighbor)	$g(\mathcal{B}) \rightarrow$ one value per window
Output	binary code $\rightarrow$ histogram	binary code $\rightarrow$ histogram (same)

Limitation: Can only discover LBP-like descriptors (not gradient-based or frequency-based features).

Search Through Pipeline Structures (Fan et al., IEEE TEVC 2023)

Given fixed pools of operators, search for the **optimal pipeline structure**:

Pool	Elements
ROI selection	ROI_k : crop region k from image I
$\mathcal{P}$ -pool (12 filters)	Gaussian, Sobel, Median, Laplacian, Min, Max, ...
$\mathcal{F}$ -pool (5 extractors)	HOG, LBP, SIFT, Histogram, DIF

GP searches for which operators to apply, in what order, on which regions:

$$\hat{y} = C\left(\underbrace{\mathcal{F}_i\left(p_j(\text{ROI}_k(I))\right)}_{\text{branch 1}} \oplus \underbrace{\mathcal{F}_{i'}\left(p_{j'}(\text{ROI}_{k'}(I))\right)}_{\text{branch 2}} \oplus \dots\right)$$

Example: Sobel $\rightarrow$ HOG on one ROI, Gaussian $\rightarrow$ LBP on another, concatenate and classify.

Limitation: all $\mathcal{F}_i$ are existing hand-designed extractors. Cannot invent new $\mathcal{F}$ beyond the pool.

Approach 2: Program Synthesis with LLMs

Painpoint of Approach 1: Defining a universal basis pool that covers all feature types (gradient, frequency, statistical, ...) is impractical, and combinatorial search over large pools is intractable.

Alternative: Let an LLM directly **generate code** that implements a feature extractor:

$$\mathcal{F} : \mathbb{R}^{h \times w} \rightarrow \mathbb{R}^d, \quad \text{represented as a Python function } f(\mathbf{X})$$

- **No fixed basis pool:** the function body can use any computation (loops, conditionals, library calls), so the space of expressible features is unbounded
- **LLM as search engine:** instead of blind GP mutation, the LLM draws on its training corpus (textbooks, papers, code) to generate functions that implement known concepts in novel combinations

Example LLM-generated $\mathcal{F}$:

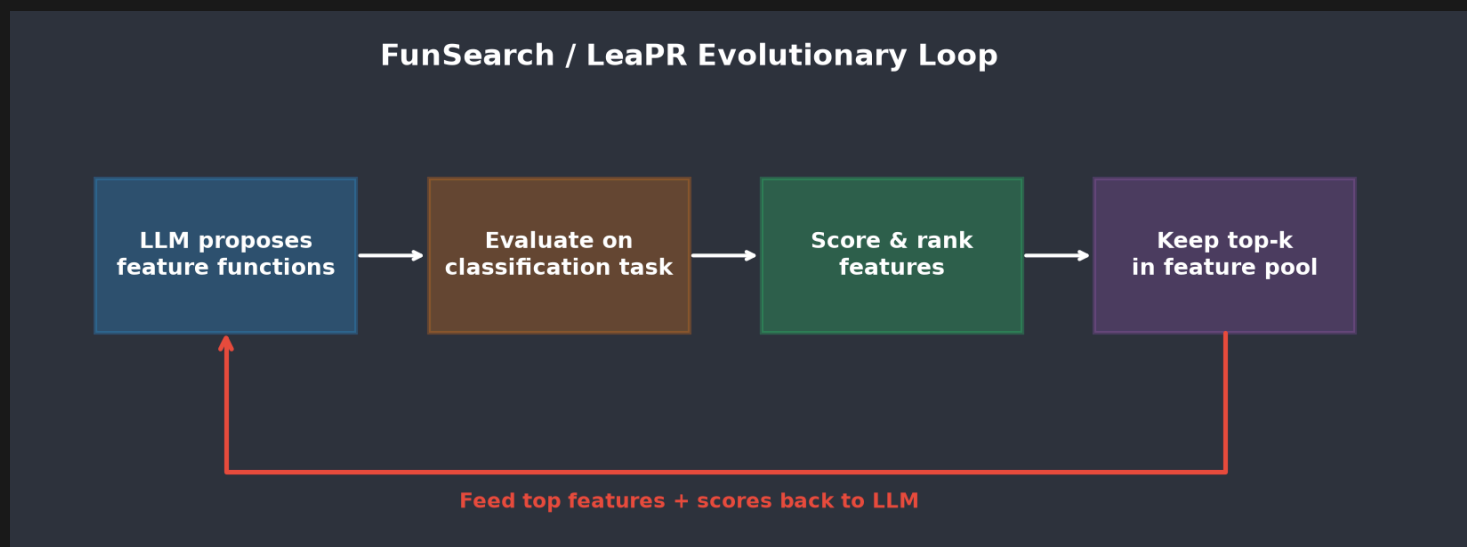
```
def feature(region):
    edges = sobel(region)
    return [np.mean(edges),
            np.std(region) / (np.mean(region) + 1e-6),
            entropy(histogram(region))]
```

The LLM “knows” that edge magnitude, contrast ratio, and texture entropy are useful features, and composes them into one extractor.

	Approach 1 (GP)	Approach 2 (LLM)
Search space	fixed pool	open-ended code
Guidance	fitness only	domain knowledge + fitness

Approach 2: Program Synthesis with LLMs

FunSearch (DeepMind, Nature 2024): evolutionary loop driven by LLM instead of blind GP.



LLM iteratively proposes Python functions → evaluate on downstream task → best kept in a pool → LLM sees top performers and proposes improvements.

LeaPR — F2: Features FunSearch (Poesia & Sampaio, Preprint 2025)

LeaPR applies the FunSearch loop to image feature discovery. Each feature is a programmatic function $f_i : \mathbf{X} \rightarrow \mathbb{R}$, synthesized by an LLM.

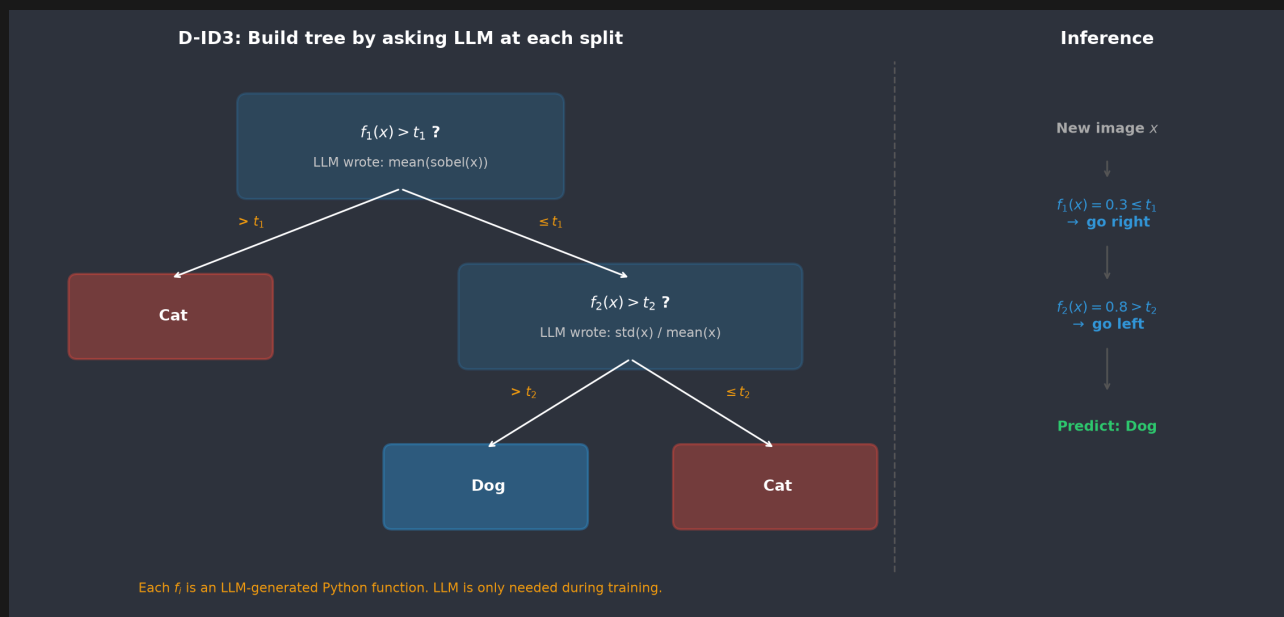
F2 = FunSearch adapted for features (black-box):

1. Start with an initial feature pool $\{f_1, \dots, f_n\}$
2. Train a Random Forest classifier on current features
3. Extract feature importance weights $\rightarrow$ LLM sees which f_i are useful vs. redundant
4. LLM proposes new f_{n+1} to **complement** the pool (not duplicate)
5. Repeat: 100 iterations $\times$ 10 features = 1000 candidates

The classifier is a black box: F2 does not control **how** features are used, only **which** features exist.

LeaPR — D-ID3: Dynamic ID3 (Poesia & Sampaio, Preprint 2025)

D-ID3 builds a decision tree where each split uses an LLM-generated feature $f_i : \mathbb{R}^{h \times w} \rightarrow \mathbb{R}$:



At each node with mixed classes, the LLM generates a new f_i tailored to those samples. The tree itself is the classifier: no separate SVM/RF needed. **On MNIST: 96.91% accuracy.**

The Problem with Code Generation

Example LLM-generated feature $f_i : \mathbb{R}^{28 \times 28} \rightarrow \mathbb{R}$ from LeaPR on MNIST:

```
def feature_42(image):  
    top = image[:14, :]  
    bottom = image[14:, :]  
    return np.std(top) / (np.mean(bottom) + 1e-6)
```

You can read the code, but:

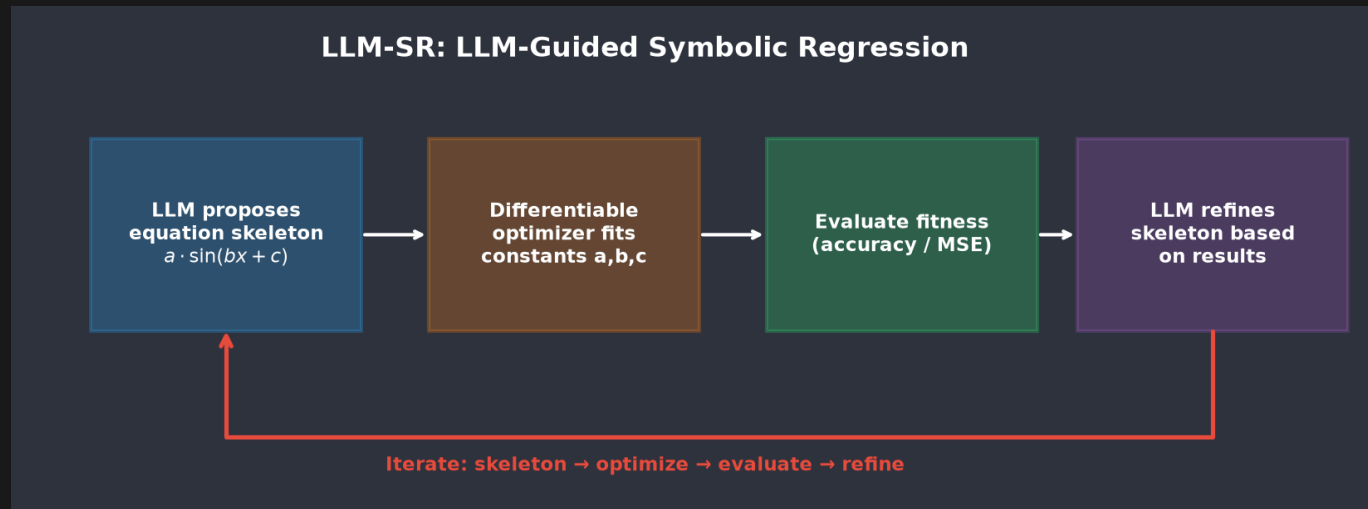
- Is it rotation invariant? **Unknown**
- Is it noise robust? **Unknown**
- Can you transfer it to another task? **No**
- Can you derive its mathematical properties? **No**

Code readability $\neq$ mathematical interpretability. LeaPR's features are readable but not mathematically analyzable. We want formula output instead.

Papadimitriou et al. "Programmatic Representation Learning with Language Models" Preprint, 2023.

Direction 2b: LLM-Guided Symbolic Regression

Key insight: Instead of letting the LLM write arbitrary code, constrain the output to **mathematical expressions**, and use differentiable optimization to fit constants.



Output is a **compact mathematical formula**, not a program. Formulas can be analyzed for invariance, robustness, and transferability.

LLM-SR (Shojaee et al., ICLR 2025, Oral)

LLM proposes an **equation skeleton** with placeholder constants; a differentiable optimizer fills them:

1. LLM proposes skeleton: $f(x) = a \cdot \sin(bx + c) + d \cdot x^p$
2. Differentiable optimizer fits constants (a, b, c, d, p) to data
3. Evaluate fit $\rightarrow$ feed error + current skeleton back to LLM
4. LLM refines the skeleton structure, repeat



Applied to physics, biology, materials science. **Not yet applied to images.**

LaSR (Grayeli et al., NeurIPS 2024)

LaSR augments classical symbolic regression (PySR) with LLM-guided mutation:

- Standard PySR: GP evolves math expressions via random mutation + crossover
- LaSR adds: with ~1% probability, replace GP mutation with an **LLM-guided mutation**
 - LLM sees the current expression + fitness
 - Proposes a semantically meaningful modification (not random)
- LLM also discovers a **concept library**: textual abstractions like “exponential decay” or “inverse square law” that guide future mutations

PySR (GP) $\xrightarrow[\text{1\% of mutations}]{\text{LLM}}$ semantically guided modification

Applied to Feynman equations. **Not yet applied to images.**

The Gap

Research Gap: No LLM-guided math-formula image descriptor discovery

Method	Search	Operator Level	Output	Applied to Images?
GP-criptor (2017)	Blind GP	Pixel-level (min/max/mean/std)	Math formula	Yes
LeaPR (2025)	LLM	Arbitrary (Python code)	Code	Yes
LLM-SR (2025)	LLM-guided	Configurable	Math formula	No (physics only)
LaSR (2024)	LLM + PySR	Configurable	Math formula	No (physics only)
Ours (proposed)	LLM-guided	Expanded pixel-level	Math formula	Yes

No one has applied LLM-guided symbolic regression to discover mathematically interpretable image feature descriptors from pixel-level operations.

Proposed Research Plan

Three levels of increasing explainability, each building on the previous:

	Approach	Output	Explainability
Baseline	FunSearch/LeaPR: LLM generates feature as code	Python function	Low
Step 2	LLM-SR: LLM generates feature as math formula	$\mathcal{F}_{n+1}^* = a \cdot \nabla^2 I + b \cdot \frac{\sigma(w)}{\mu(w)}$	Medium
Step 3	LLM decomposes existing descriptors into shared basis	$\mathcal{F}_i = g_i(\phi_1, \dots, \phi_m)$	High

All three plug into the same evaluation pipeline:

$$I \xrightarrow{\text{ROI}} \mathbf{X} \xrightarrow{\mathcal{P}} \mathbf{X}' \xrightarrow{\mathcal{F}_1, \dots, \mathcal{F}_n, \mathcal{F}_{n+1}^*} [\mathbf{f}_1 \oplus \dots \oplus \mathbf{f}_n \oplus \mathbf{f}_{n+1}^*] \xrightarrow{\mathcal{C}} \hat{y}$$

Baseline validates the pipeline works; **Step 2** adds mathematical interpretability; **Step 3** aims to discover a principled basis pool $\{\phi_k\}$ such that $\mathcal{F}_i \approx g_i(\phi_1, \dots, \phi_m)$ for all existing descriptors.

References

1. Al-Sahaf et al., “Automatically Evolving Rotation-Invariant Texture Image Descriptors by Genetic Programming,” *IEEE Trans. Evolutionary Computation*, vol. 21, no. 1, pp. 83–101, 2017.
2. Fan et al., “Genetic Programming for Image Classification: A New Program Representation With Flexible Feature Reuse,” *IEEE Trans. Evolutionary Computation*, vol. 27, no. 3, pp. 460–474, 2023.
3. Romera-Paredes et al., “Mathematical Discoveries from Program Search with Large Language Models,” *Nature*, vol. 625, pp. 468–475, 2024.
4. Poesia & Sampaio, “Programmatic Representation Learning with Language Models,” *arXiv:2510.14825*, 2025.
5. Shojaee et al., “LLM-SR: Scientific Equation Discovery via Programming with Large Language Models,” *ICLR (Oral)*, 2025.
6. Grayeli et al., “Symbolic Regression with a Learned Concept Library,” *NeurIPS*, 2024.
7. Ojala et al., “A Comparative Study of Texture Measures with Classification Based on Feature Distributions,” *Pattern Recognition*, vol. 29, no. 1, pp. 51–59, 1996.